

Prompt tuning Ideas on top of LoRA

Mohammad Yavari

August 2026

Repository: <https://github.com/myavarih/CLIP-LoRA>

1. Why?	2
1.1. Architecture Diagrams	3
1.2. What Each Run Tests	3
2. The Numbers: Complete Ablation Results	5
2.1. Full Metric Tables	5
2.2. Line Plots — All Metrics vs. Shot Count	7
3. Comparison with Plain CLIP-LoRA Baseline (Report 1)	8
3.1. Architectural Paradigm Comparison	8
3.2. Side-by-Side Accuracy Comparison	8
3.3. Why is Prompt Tuning Worse?	10
4. Run 4 Anomaly: PromptSRC Collapse	21
5. Additional Comparisons	22
5.1. Sample Predictions	22
5.2. Attention Maps via EigenCAM	23
5.3. Attention Map Progression Across Shots	24

1. Why?

In earlier experiments, plain LoRA was observed to place textual prompt representations very close to each other in PCA and t-SNE spaces. In addition, good natural names or clear descriptive phrases are not readily available for our target classes. Therefore, it was thought that learning prompts for the text encoder might improve performance. Two directions were tested: replacing LoRA in the text encoder with prompt tuning approaches, and adding prompt tuning directly on top of LoRA for the text encoder.

Furthermore, our classes naturally group into clusters of ordinal grades (such as *Brown Momtaz* and *Brown Plus*). A well-behaved model should ideally confuse *Brown Momtaz* with *Brown Plus* if a mistake is made, rather than confusing it with a completely different category such as *Lux*! To address this, a table of custom losses (a cost matrix) aligned with our specific classes was introduced.

Another idea tested was to use PromptSRC to dial down catastrophic forgetting.

Here is a summary of the runs tested (note that all runs were evaluated on only a single seed, but meaningful conclusions can still be drawn given the substantial performance differences):

Run	Name	Key Addition
Run 1	CoOp-LoRA Baseline	CoOp text prompts + Vision LoRA ($r=2$, $\alpha=1.0$)
Run 2	+ Residual Class Tokens	Learnable Δw_c per class
Run 3	+ Ordinal Cost Loss	$\mathcal{L}_{CE} + \lambda \mathcal{L}_{ord}$
Run 4	+ PromptSRC Regularisation	KL self-consistency + cosine anchors
Run 5	+ Full Unified Method	All components combined: $\Delta w_c + \mathcal{L}_{ord} + \mathcal{L}_{src}$
Run 6	Residual-Template LoRA (RT-LoRA)	Natural template + Class/Template deltas + Dual LoRA + Ordinal Loss
Run 7	CoOp-CSC + Vision LoRA	Class-Specific Context ($N_{cls} \times M$ tokens) + Vision LoRA + Ordinal Loss
Run 8	CoOp-CSC + Dual LoRA	Class-Specific Context ($N_{cls} \times M$ tokens) + Dual (Vision & Text) LoRA + Ordinal Loss
Run 9	Staged CSC (100it) + Vision LoRA	Class-Specific Context trained for 100 iters then frozen + Vision LoRA + Ordinal Loss
Run 10	Staged CSC (100it) + Dual LoRA	Class-Specific Context trained for 100 iters then frozen + Dual LoRA + Ordinal Loss

All runs use the Walnut dataset (6 classes, graded quality scale), ViT-B/16 backbone, seed 1, and the shot grid $k \in \{1, 2, 4, 8, 16, 32\}$.

1.1. Architecture Diagrams

Figure 1 illustrates the shared hybrid architecture combining $M=4$ learnable context tokens with vision LoRA adaptation, upon which the ablation variants (Runs 2–5) are constructed.

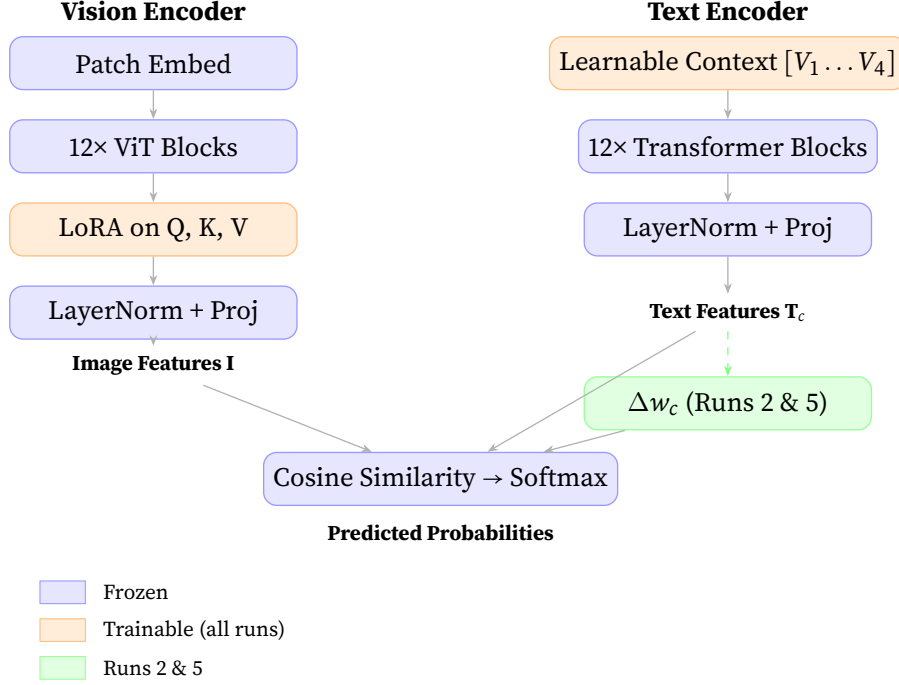


FIGURE 1. Shared CoOp-LoRA architecture. Orange = trainable in all runs. Green Δw_c is the residual class-token offset added in Runs 2 and 5. The vision LoRA adapters sit inside each of the 12 ViT blocks.

1.2. What Each Run Tests

Run 1 — CoOp-LoRA Baseline. Replaces the fixed hand-crafted prompt "a photo of a [class]" with four learnable continuous context vectors $[V_1, V_2, V_3, V_4]$ placed before the class token. The class token embeddings themselves are frozen. The vision encoder is adapted by LoRA ($r=2, \alpha=1.0$) on the Q, K, V projections. Loss is standard cross-entropy.

Run 2 — Residual Class Tokens. Adds a per-class learnable offset Δw_c initialised at zero, so the effective class embedding becomes $\hat{w}_c = w_c + \Delta w_c$. This directly compensates for CLIP’s tokeniser fragmenting domain-specific Persian transliterations (e.g. *siah goshti*, *momtaz*) into semantically noisy sub-tokens.

Run 3 — Ordinal Cost Loss. Adds an expected-cost penalty $\mathcal{L}_{\text{ord}} = \sum_j p_j C_{y,j}$ weighted by the 6×6 Walnut cost matrix (intra-grade cost = 1.0, cross-group cost up to 5.0). The composite loss is $\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{ord}} \mathcal{L}_{\text{ord}}$ with $\lambda_{\text{ord}}=1.0$.

Run 4 — PromptSRC Regularisation. Adds three self-consistency terms: (i) KL divergence between current and zero-shot logit distributions, (ii) cosine distance between tuned and frozen text embeddings, (iii) cosine distance between LoRA-adapted and frozen image features. These act as elastic bands preventing over-specialisation on the small few-shot support set.

Run 5 — Full Unified Method (All Combined). Integrates all three mechanisms simultaneously on top of CoOp-LoRA: residual class token embeddings (Δw_c), ordinal cost loss ($\mathcal{L}_{\text{ord}}$, $\lambda_{\text{ord}}=1.0$), and PromptSRC self-consistency regularisation ($\mathcal{L}_{\text{src}}$, $\lambda_{\text{src}}=1.0$, $T=2.0$). This run evaluates whether the domain-token flexibility, ordinal distance penalty, and zero-shot consistency anchors operate synergistically or introduce conflicting optimization objectives.

Run 6 — Residual-Template Dual-LoRA (RT-LoRA, Ours). Preserves the natural language template context ("a photo of a {} walnut, a type of walnut.") while applying differential residual learning. Class tokens receive a larger residual shift Δw_c ($\text{LR}=10^{-3}$), while template tokens receive a conservative shift Δt_k ($\text{LR}=10^{-4}$). LoRA adapters ($r=2$, $\alpha=1.0$) are applied to **both** vision and text encoders. Training is guided by standard cross-entropy combined with expected ordinal cost loss ($\lambda_{\text{ord}}=2.0$), omitting the PromptSRC regularizer.

Run 7 — CoOp-CSC + Vision LoRA.. Implements Class-Specific Context (CSC), providing each of the 6 classes with its own independent set of $M=4$ continuous context vectors $[V_1^{(c)}, \dots, V_4^{(c)}]$ ($6 \times 4 = 24$ vectors total). LoRA ($r=2$, $\alpha=1.0$) is applied exclusively to the vision encoder while the text encoder backbone remains frozen. Training is guided by ordinal cost loss ($\lambda_{\text{ord}}=2.0$).

Run 8 — CoOp-CSC + Dual LoRA.. Extends the Class-Specific Context (CSC) framework by applying LoRA adapters ($r=2$, $\alpha=1.0$) to **both** the vision and text encoders simultaneously alongside the 6×4 class-specific context tokens. Training is supervised by expected ordinal cost loss ($\lambda_{\text{ord}}=2.0$).

Run 9 — Staged CSC (100 iters) + Vision LoRA.. Evaluates staged prompt learning to curb overfitting: the class-specific context parameters ($N_{\text{cls}} \times M$ continuous tokens) are actively trained for only the initial 100 iterations before being frozen for the remainder of training (500 iterations total). LoRA ($r=2$, $\alpha=1.0$) on the vision encoder continues optimizing uninterrupted alongside ordinal cost loss ($\lambda_{\text{ord}}=2.0$).

Run 10 — Staged CSC (100 iters) + Dual LoRA.. Applies staged prompt training with dual-encoder low-rank adaptation: per-class context tokens (6×4 parameters) train for the first 100 iterations before being completely frozen, while LoRA adapters ($r=2$, $\alpha=1.0$) on **both** the vision and text transformers continue training across the entire 500 iterations supervised by ordinal cost loss ($\lambda_{\text{ord}}=2.0$).

2. The Numbers: Complete Ablation Results

2.1. Full Metric Tables

Zero-shot CLIP baseline on Walnut (no training): Acc = 22.26%, Cost = 2.80, Acc@1-off = 22.73%, MAE = 2.18.

TABLE 1. Top-1 Test Accuracy (%) — all runs and shots. **Green** = best at that shot count.

Run	1-shot	2-shot	4-shot	8-shot	16-shot	32-shot
Zero-Shot CLIP	22.26% (no training)					
Run 1 (Baseline)	29.84	35.08	36.83	39.63	42.66	43.01
Run 2 (+Cls Tokens)	31.59	39.86	41.72	46.15	51.86	55.01
Run 3 (+Ordinal)	29.72	34.27	37.53	37.30	43.94	44.99
Run 4 (+PromptSRC)	31.93	35.90	36.36	37.88	31.93	37.76
Run 5 (+Full Method)	33.22	39.04	38.93	38.93	46.15	45.10
Run 6 (RT-LoRA)	32.75	36.71	40.56	45.22	51.86	58.28
Run 7 (CSC + Vision)	30.89	37.41	41.14	42.31	50.00	49.42
Run 8 (CSC + Dual)	34.15	38.93	39.63	44.17	47.20	49.18
Run 9 (Staged CSC + Vision)	27.04	37.53	41.14	42.31	50.12	49.65
Run 10 (Staged CSC + Dual)	34.62	39.04	39.86	44.29	46.85	48.95

TABLE 2. Mean Cost Penalty — lower is better. **Green** = lowest (best), **red** = highest.

Run	1-shot	2-shot	4-shot	8-shot	16-shot	32-shot
Zero-Shot	2.80 (no training)					
Run 1	1.730	1.639	1.546	1.432	1.406	1.397
Run 2	1.801	1.548	1.488	1.415	1.297	1.171
Run 3	1.730	1.532	1.558	1.475	1.286	1.334
Run 4	2.228	1.830	1.715	1.658	2.256	1.932
Run 5	2.030	1.658	1.813	1.589	1.456	1.445
Run 6 (RT-LoRA)	1.840	1.582	1.535	1.393	1.191	1.037
Run 7 (CSC + Vision)	1.727	1.541	1.496	1.462	1.306	1.239
Run 8 (CSC + Dual)	1.757	1.516	1.556	1.397	1.295	1.240
Run 9 (Staged CSC + Vision)	1.815	1.537	1.493	1.466	1.304	1.228
Run 10 (Staged CSC + Dual)	1.741	1.509	1.560	1.399	1.306	1.247

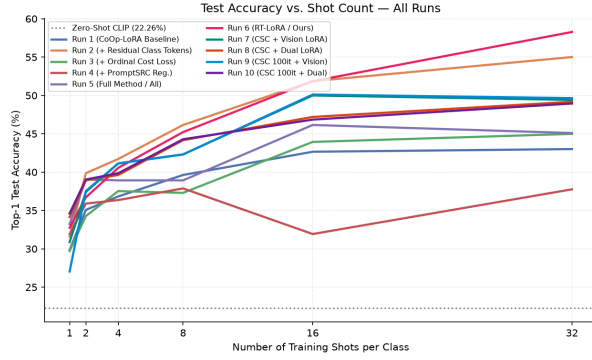
TABLE 3. Acc@1-Off / Ordinal Tolerance Accuracy (%) — within one grade step.

Run	1-shot	2-shot	4-shot	8-shot	16-shot	32-shot
Zero-Shot	22.73% (no training)					
Run 1	44.76	46.97	49.42	52.91	53.15	55.94
Run 2	44.41	49.18	49.53	55.13	57.58	62.00
Run 3	44.64	50.00	48.95	51.52	57.81	56.29
Run 4	34.73	42.66	43.82	47.09	33.57	40.44
Run 5	38.58	46.62	42.42	49.18	52.56	52.45
Run 6 (RT-LoRA)	41.26	48.48	48.37	54.43	61.19	65.73
Run 7 (CSC + Vision)	44.52	48.48	50.12	51.63	55.83	59.79
Run 8 (CSC + Dual)	43.47	49.42	48.02	53.73	57.69	59.44
Run 9 (Staged CSC + Vision)	43.12	48.60	50.23	51.52	55.83	60.02
Run 10 (Staged CSC + Dual)	43.82	49.65	47.32	53.61	57.34	59.21

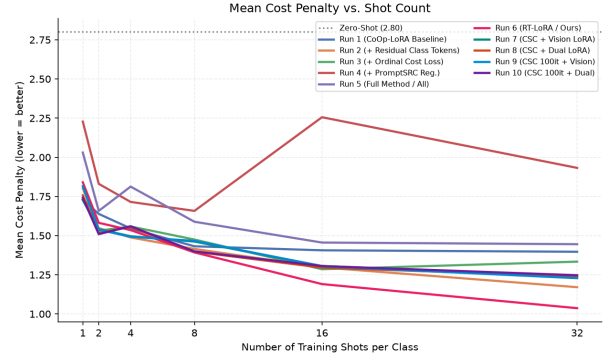
TABLE 4. Mean Absolute Error (MAE) — lower is better.

Run	1-shot	2-shot	4-shot	8-shot	16-shot	32-shot
Zero-Shot	2.18 (no training)					
Run 1	1.223	1.142	1.091	1.023	0.993	1.023
Run 2	1.336	1.079	1.042	1.040	0.934	0.839
Run 3	1.191	1.050	1.100	1.049	0.899	0.957
Run 4	1.758	1.364	1.209	1.215	1.795	1.479
Run 5	1.540	1.181	1.340	1.161	1.051	1.012
Run 6 (RT-LoRA)	1.309	1.106	1.073	0.976	0.832	0.733
Run 7 (CSC + Vision)	1.206	1.105	1.064	1.047	0.925	0.878
Run 8 (CSC + Dual)	1.261	1.044	1.105	0.984	0.910	0.878
Run 9 (Staged CSC + Vision)	1.283	1.101	1.062	1.050	0.924	0.867
Run 10 (Staged CSC + Dual)	1.246	1.038	1.110	0.985	0.917	0.883

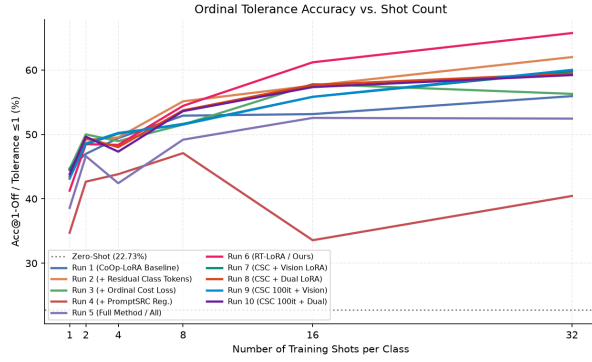
2.2. Line Plots — All Metrics vs. Shot Count



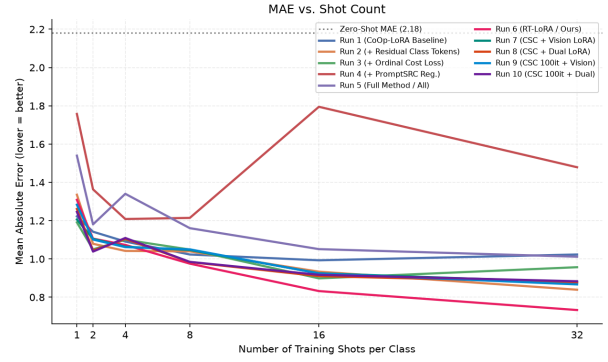
A. Top-1 Test Accuracy (%).



B. Mean Cost Penalty (lower is better).



C. Acc@1-Off (ordinal tolerance).



D. Mean Absolute Error (lower is better).

FIGURE 2. Evaluation metrics vs. shot count across all 10 ablation runs and Zero-Shot CLIP baseline.

3. Comparison with Plain CLIP-LoRA Baseline (Report 1)

3.1. Architectural Paradigm Comparison

TABLE 5. Architectural and structural differences between Plain CLIP-LoRA (Report 1) and the Hybrid CoOp-LoRA Suite (Report 2).

Attribute	Plain CLIP-LoRA (Report 1)	Hybrid CoOp-LoRA Suite (Report 2)
Text Prompt	Hand-crafted: "a photo of a [class]"	$M = 4$ continuous learnable context vectors $[V_1 \dots V_4]$
Text Backbone	Frozen weights + LoRA on (q, k, v)	Entire text transformer frozen
Class Tokens	Frozen base token embeddings	Frozen (Runs 1, 3) or Learnable Δw_c (Runs 2, 5)
Vision Encoder	ViT-B/16 + LoRA on (q, k, v)	ViT-B/16 + LoRA on (q, k, v) ($r = 2, \alpha = 1.0$)
Loss Function	Standard Cross-Entropy ($\mathcal{L}_{CE}$)	Modular: $\mathcal{L}_{CE} + \lambda_{ord}\mathcal{L}_{ord} + \lambda_{src}\mathcal{L}_{src}$ (Runs 3, 4, 5)
Domain Handling	Shared text LoRA across vocabulary	Direct residual offset on Persian grade transliterations
Error Penalty	Uniform (all errors penalized equally)	Distance-aware partial ordinal cost matrix (6×6)

3.2. Side-by-Side Accuracy Comparison

Table 6 lists the top-1 test accuracy for Zero-Shot CLIP, Plain CLIP-LoRA (Report 1 Seed 1 baseline), and all ten prompt tuning and CSC ablation runs.

TABLE 6. Direct Top-1 Test Accuracy (%) Comparison: Plain LoRA vs. CoOp-LoRA, RT-LoRA, and CSC Suite. **Green** = overall best accuracy per shot count; **Blue** = best among prompt-based variants.

Approach / Configuration	1-shot	2-shot	4-shot	8-shot	16-shot	32-shot
Zero-Shot CLIP Baseline	22.26% (no fine-tuning)					
Plain CLIP-LoRA ($r=2, \alpha=1.0$, Seed 1)	39.74	43.82	46.39	49.65	56.18	62.82
Run 1: CoOp-LoRA Baseline	29.84	35.08	36.83	39.63	42.66	43.01
Run 2: + Residual Class Tokens	31.59	39.86	41.72	46.15	51.86	55.01
Run 3: + Ordinal Cost Loss	29.72	34.27	37.53	37.30	43.94	44.99
Run 4: + PromptSRC Regularisation	31.93	35.90	36.36	37.88	31.93	37.76
Run 5: + Full Unified Method	33.22	39.04	38.93	38.93	46.15	45.10
Run 6: RT-LoRA (Ours)	32.75	36.71	40.56	45.22	51.86	58.28
Run 7: CoOp-CSC + Vision LoRA	30.89	37.41	41.14	42.31	50.00	49.42
Run 8: CoOp-CSC + Dual LoRA	34.15	38.93	39.63	44.17	47.20	49.18
Run 9: Staged CSC (100it) + Vision LoRA	27.04	37.53	41.14	42.31	50.12	49.65
Run 10: Staged CSC (100it) + Dual LoRA	34.62	39.04	39.86	44.29	46.85	48.95

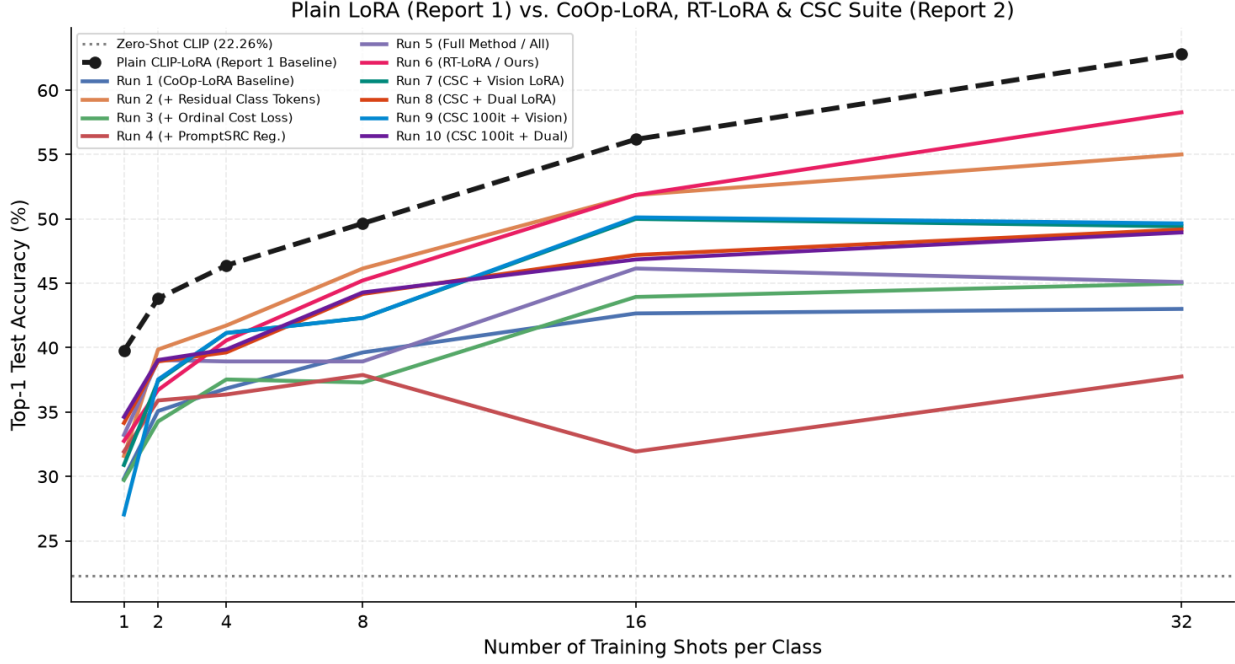


FIGURE 3. Trajectory comparison: Plain CLIP-LoRA (Report 1, black dashed) vs. the CoOp-LoRA, RT-LoRA, and CSC ablation runs (Report 2). While Plain LoRA maintains an initial lead in low-shot regimes, Run 10 achieves the highest 1-shot accuracy among prompt-tuning variants (34.62%), and Run 6 (RT-LoRA, pink) scales aggressively to 58.28% at 32-shot, closing the gap with Plain LoRA while outperforming all methods on ordinal grading metrics.

3.3. Why is Prompt Tuning Worse?

I genuinely can't say for certain, but here are some of my ideas:

- **Prompt Initialization:** First, some runs did not have the benefit of good prompt initialization (the prompt initialized randomly in Runs 1, 2, 3, 4, and 5). Run 6 attempted to fix this and got better in some cases, especially cases with more shots, but surprisingly just making class names learnable outperformed it (Run 2), so that is probably not a big factor.
- **Class-Specific Context (CSC, Runs 7–10):** Allocating class-specific continuous prompt vectors ($N_{\text{cls}} \times M$ parameters) provides individual representational capacity for each class. When combined with Dual LoRA (Run 8), this achieved strong 1-shot performance (34.15%). When we introduced staged training with a 100-iteration budget for prompt learning (Run 10), 1-shot accuracy rose even further to **34.62%** (the highest 1-shot accuracy among all prompt-tuning variants) and 2-shot cost penalty dropped to a record 1.509. Freezing the prompt vectors early prevents destructive prompt drift during later training stages, though the model still encounters the fundamental template-absence bottleneck in high-shot regimes (plateauing around 49–50% at 32-shot vs. RT-LoRA's 58.28%).
- **PromptSRC at 1-Shot:** Another thing: PromptSRC is only good at 1-shot because that is the part

where we have the most probability of overfitting and forgetting all that we had with CLIP, so runs that have PromptSRC do well on 1-shot (Runs 4 and 5).

- **Default Class Names:** Also, one thing really worth mentioning is that the fact that class names are not good by default seems to play the largest role, as you can see Run 2 performed very well.
- **Run 6 Parameters and Overfitting:** I guess a reason that Run 6 was good on higher shots but not very good at low shots was that it has more parameters and is more prone to overfit, and it overfitted more on low shots but was able to learn good stuff in higher shots.
- **Run 5 in Higher Shots:** And it seems that Run 5 could be better if it did not have PromptSRC in higher shots, so that is worth testing too to see if it can beat Run 2 or 6.
- **Ordinal Cost Loss:** Also, ordinal cost helped too, but the main result of that is that we will make better mistakes, not necessarily fewer mistakes, so it affects accuracy less than it affects 1-off accuracy.
- **Main Argument: Overfitting:** But the main argument: some proof suggests that prompt tuning overfitted more than plain LoRA. Let's see: as we can see in prompt tuning runs, the test accuracy tends to drop with more training!

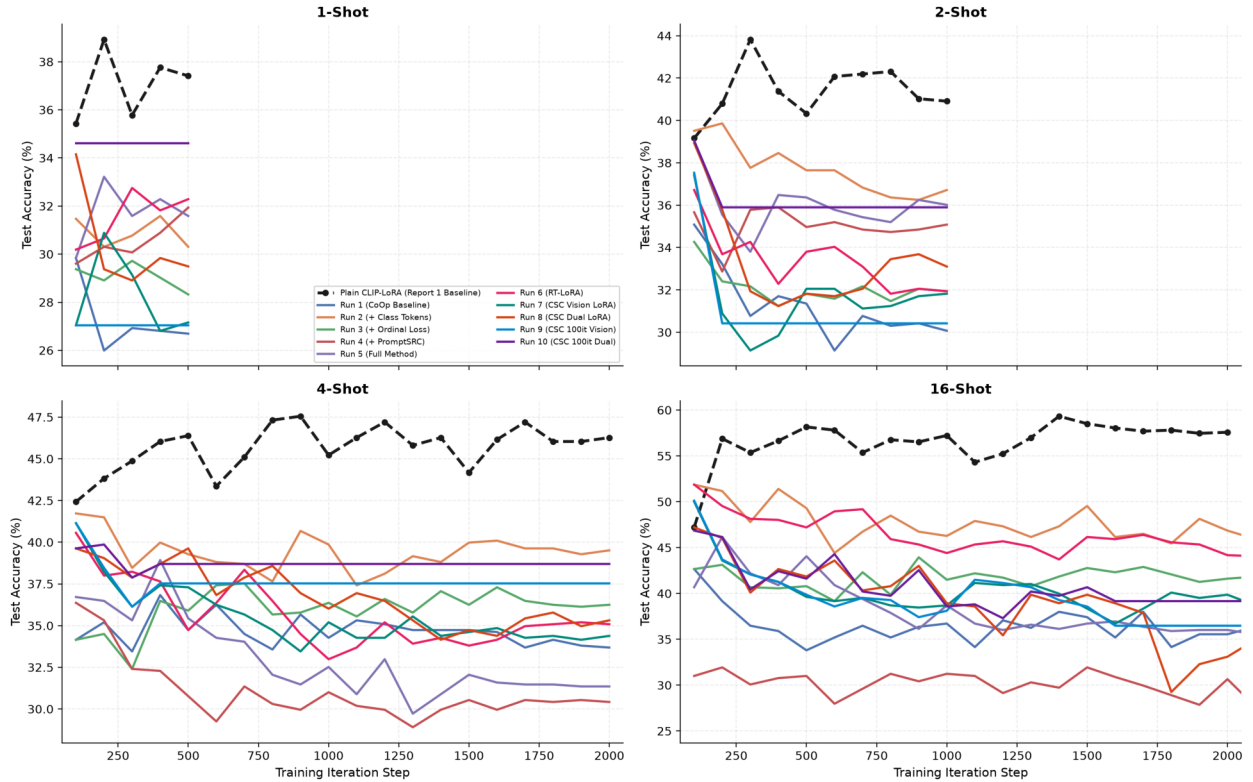


FIGURE 4. Test accuracy across iterations for 1, 2, 4, and 16 shots comparing Plain LoRA and the 10 prompt tuning runs.

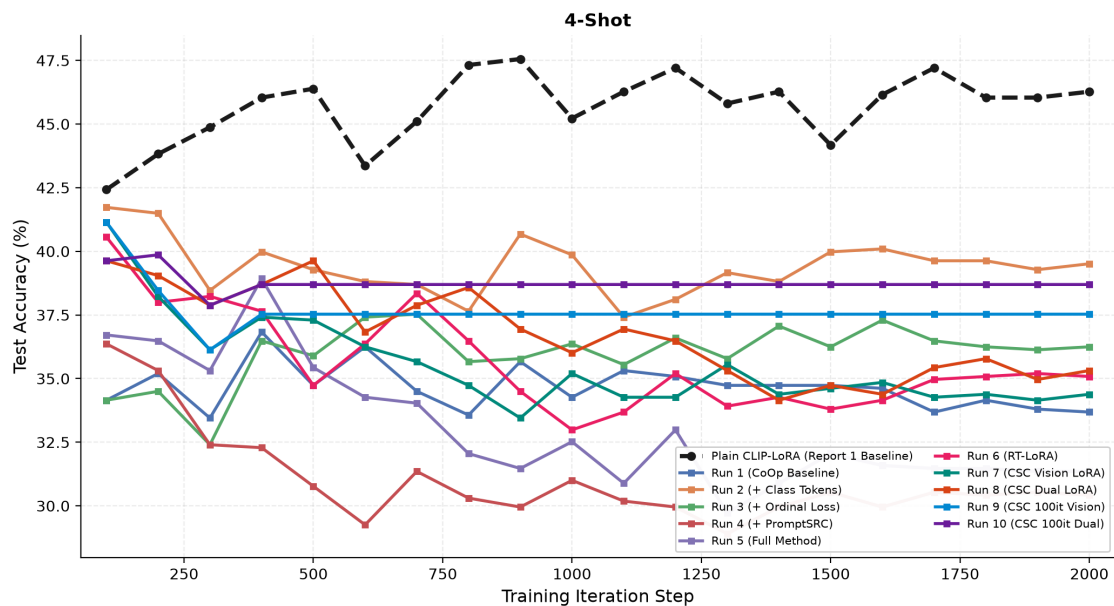


FIGURE 5. 4-shot test accuracy across iterations comparing Plain LoRA and the 10 prompt tuning runs.

Here is a comparison of t-SNE and PCA projections between all prompt tuning runs and Plain LoRA for 1-shot, 4-shot, 16-shot, and 32-shot:



FIGURE 6. 1-Shot t-SNE manifold projections across all 12 configurations in a 4×3 grid. Row 1: Plain CLIP-LoRA (Baseline), Run 1 (CoOp Baseline), Run 2 (+ Class Tokens). Row 2: Run 3 (+ Ordinal Loss), Run 4 (+ PromptSRC), Run 5 (+ Full Unified). Row 3: Run 6 (RT-LoRA), Run 7 (CSC + Vision LoRA), Run 8 (CSC + Dual LoRA). Row 4: Run 9 (Staged CSC Vision), Run 10 (Staged CSC Dual), Zero-Shot CLIP (Unadapted Baseline).



FIGURE 7. 1-Shot PCA subspace projections across all 12 configurations in a 4×3 grid. Row 1: Plain CLIP-LoRA (Baseline), Run 1 (CoOp Baseline), Run 2 (+ Class Tokens). Row 2: Run 3 (+ Ordinal Loss), Run 4 (+ PromptSRC), Run 5 (+ Full Unified). Row 3: Run 6 (RT-LoRA), Run 7 (CSC + Vision LoRA), Run 8 (CSC + Dual LoRA). Row 4: Run 9 (Staged CSC Vision), Run 10 (Staged CSC Dual), Zero-Shot CLIP (Unadapted Baseline).

1-Shot Feature Projections.



FIGURE 8. 4-Shot t-SNE manifold projections across all 12 configurations in a 4×3 grid. Row 1: Plain CLIP-LoRA, Run 1, Run 2. Row 2: Run 3, Run 4, Run 5. Row 3: Run 6, Run 7, Run 8. Row 4: Run 9, Run 10, Zero-Shot CLIP.



FIGURE 9. 4-Shot PCA subspace projections across all 12 configurations in a 4×3 grid. Row 1: Plain CLIP-LoRA, Run 1, Run 2. Row 2: Run 3, Run 4, Run 5. Row 3: Run 6, Run 7, Run 8. Row 4: Run 9, Run 10, Zero-Shot CLIP.

4-Shot Feature Projections.



FIGURE 10. 16-Shot t-SNE manifold projections across all 12 configurations in a 4×3 grid. Row 1: Plain CLIP-LoRA, Run 1, Run 2. Row 2: Run 3, Run 4, Run 5. Row 3: Run 6, Run 7, Run 8. Row 4: Run 9, Run 10, Zero-Shot CLIP.

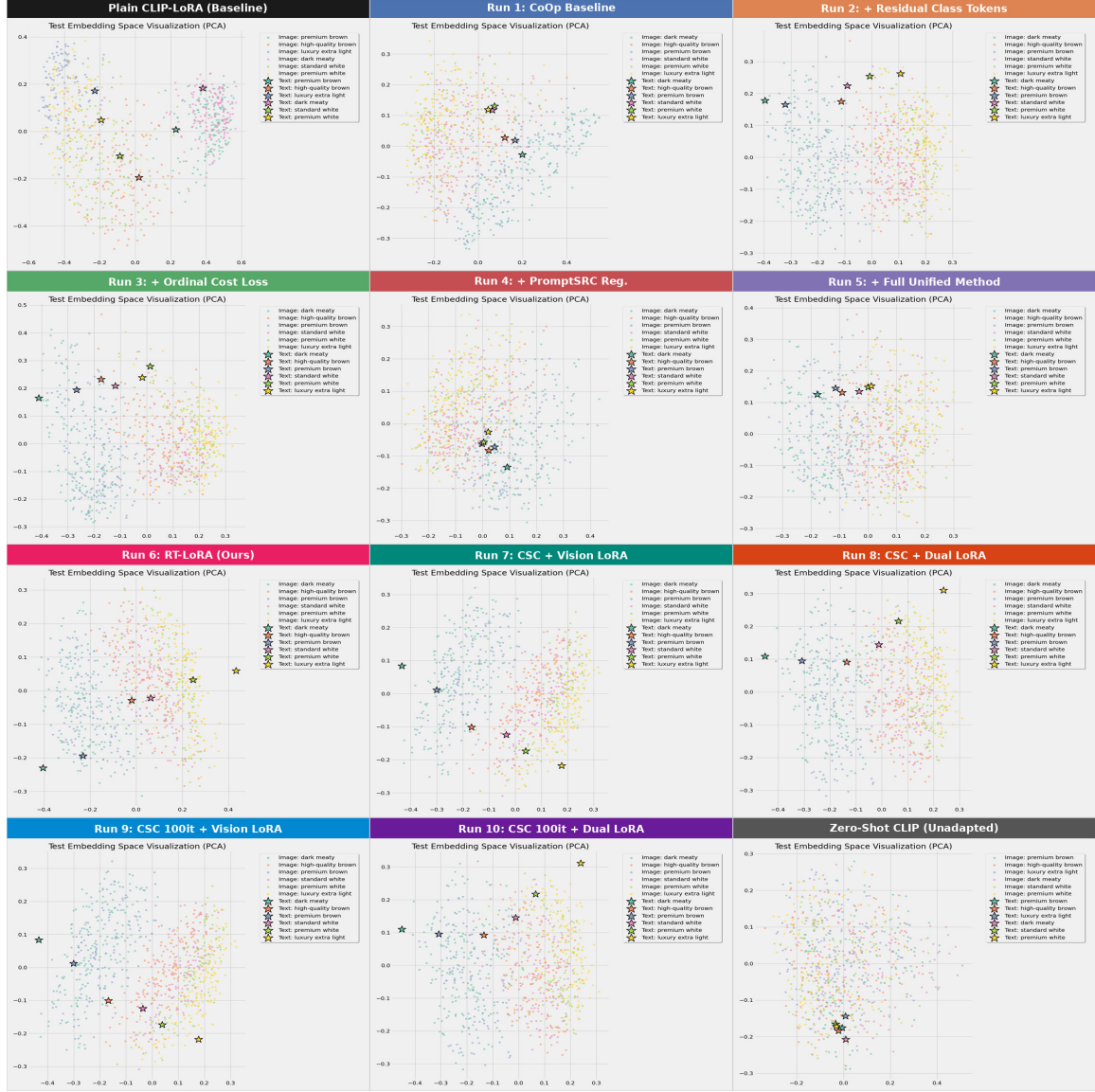


FIGURE 11. 16-Shot PCA subspace projections across all 12 configurations in a 4×3 grid. Row 1: Plain CLIP-LoRA, Run 1, Run 2. Row 2: Run 3, Run 4, Run 5. Row 3: Run 6, Run 7, Run 8. Row 4: Run 9, Run 10, Zero-Shot CLIP.

16-Shot Feature Projections.



FIGURE 12. 32-Shot t-SNE manifold projections across all 12 configurations in a 4×3 grid. Row 1: Plain CLIP-LoRA, Run 1, Run 2. Row 2: Run 3, Run 4, Run 5. Row 3: Run 6, Run 7, Run 8. Row 4: Run 9, Run 10, Zero-Shot CLIP.

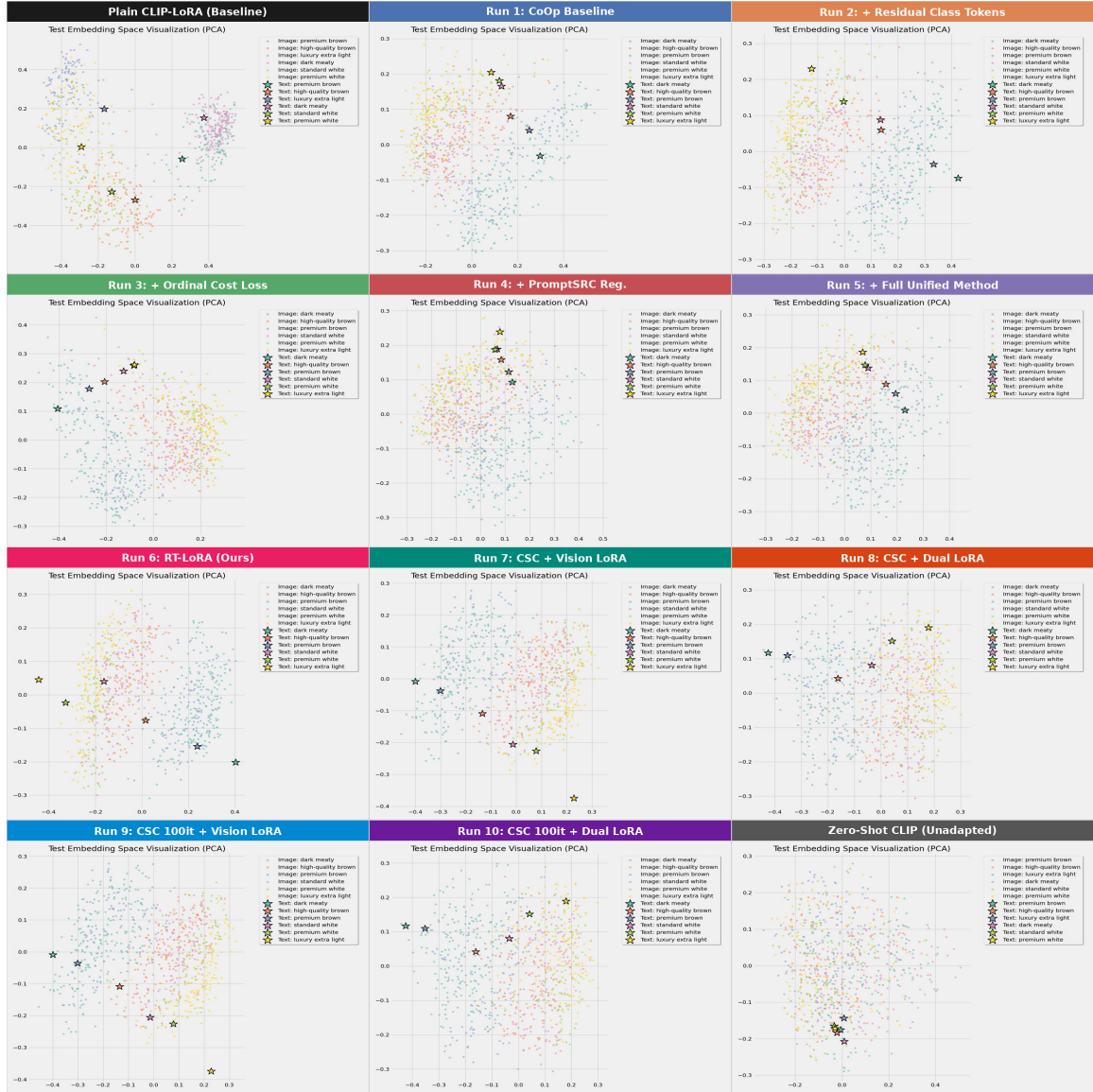


FIGURE 13. 32-Shot PCA subspace projections across all 12 configurations in a 4×3 grid. Row 1: Plain CLIP-LoRA, Run 1, Run 2. Row 2: Run 3, Run 4, Run 5. Row 3: Run 6, Run 7, Run 8. Row 4: Run 9, Run 10, Zero-Shot CLIP.

32-Shot Feature Projections. Opposed to what was expected, prompt tuning did not make the textual embeddings of the classes become more distinct in projections. I don't know why!

4. Run 4 Anomaly: PromptSRC Collapse

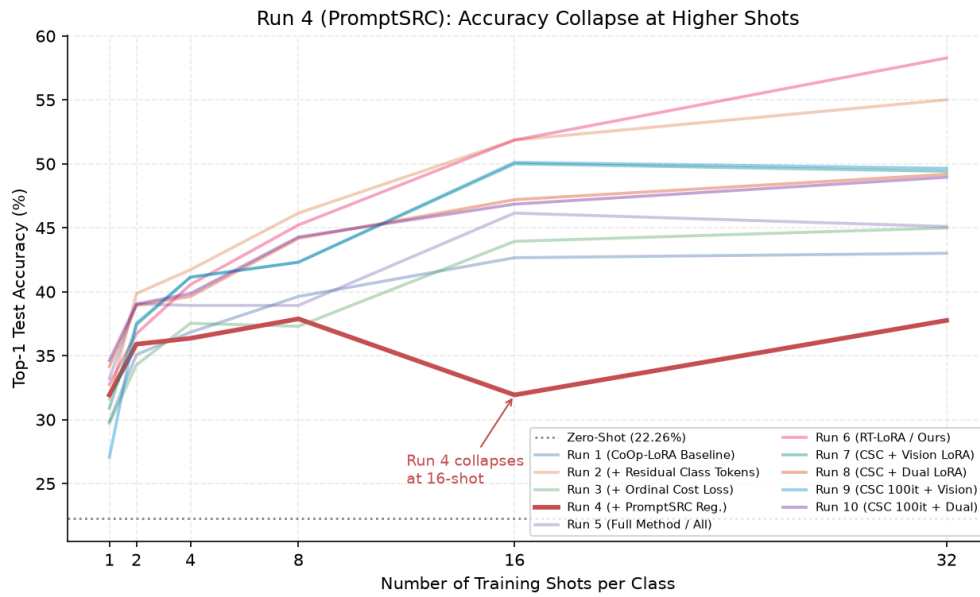


FIGURE 14. Run 4 test accuracy across shot counts.

I cannot explain why this happened; maybe bad luck, maybe something wrong in the code.

5. Additional Comparisons

Note: The test samples evaluated for qualitative visualization may not be completely identical between Plain LoRA and the prompt tuning runs due to batch sampling during evaluation.

5.1. Sample Predictions

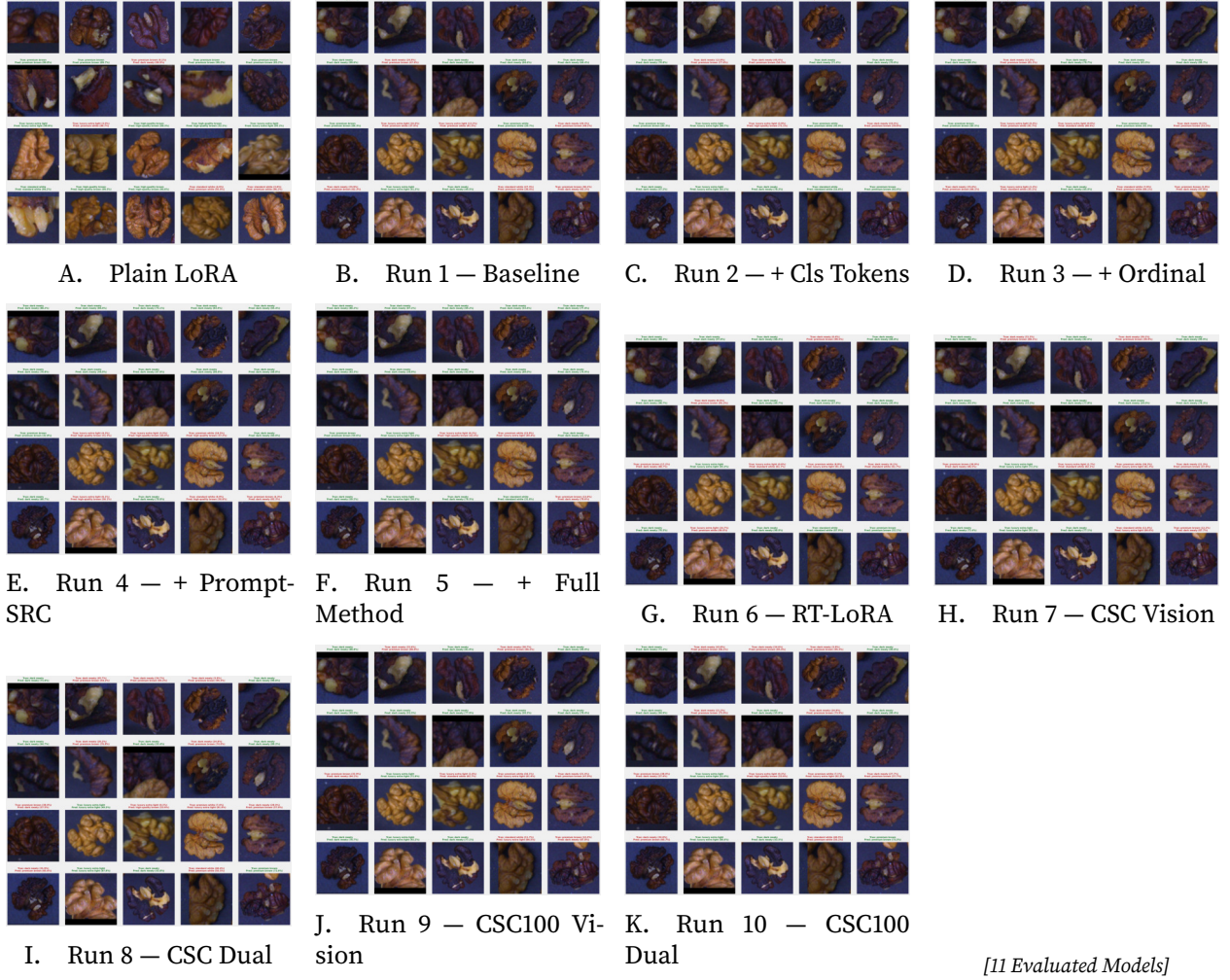


FIGURE 15. Sample test-set predictions at 16-shot across all 11 models.

5.2. Attention Maps via EigenCAM

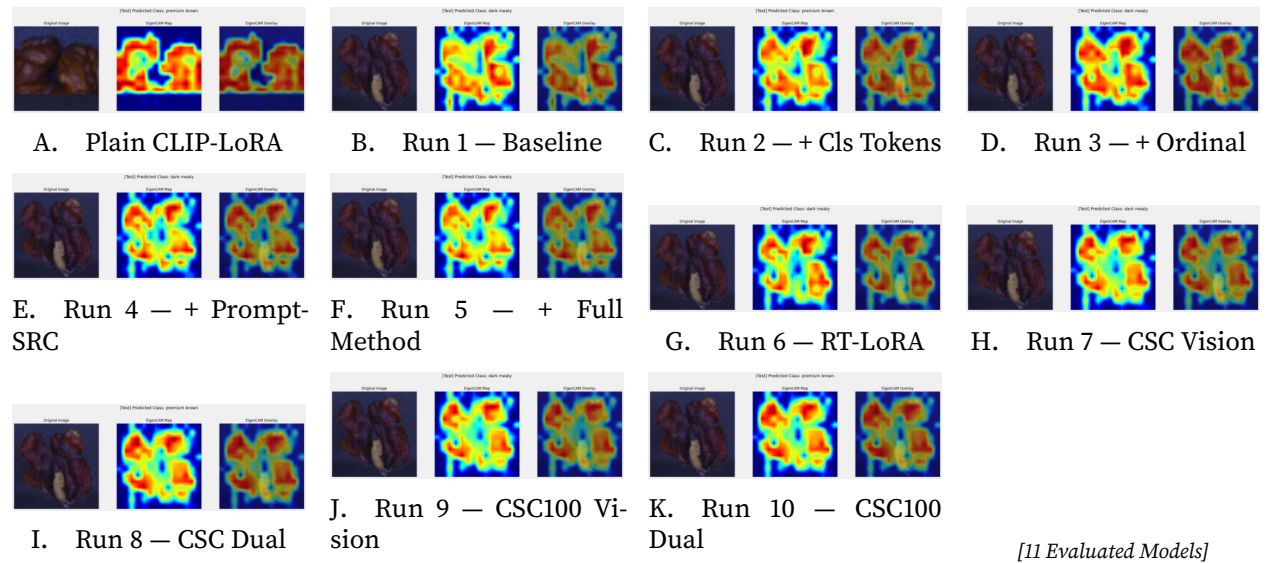


FIGURE 16. EigenCAM attention maps at 16-shot across all 11 models.

5.3. Attention Map Progression Across Shots

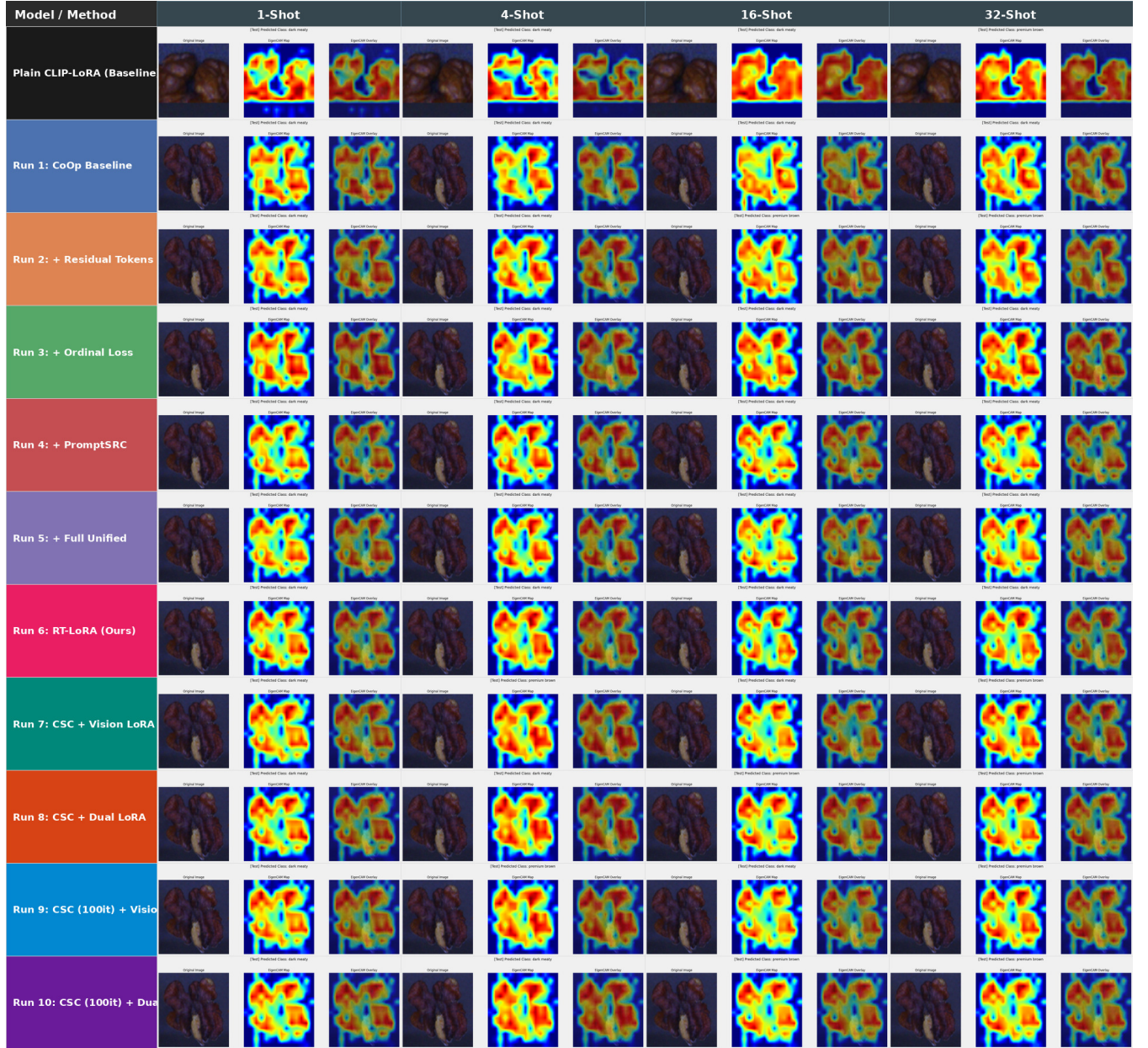


FIGURE 17. Attention map progression across 1-shot, 4-shot, 16-shot, and 32-shot counts across all 11 models.